

Name: Vũ Đại Dương

ID: 23520359

Class: IT007.P28.2

OPERATING SYSTEM LAB 5'S REPORT

SUMMARY

Task		Status	Page
Section 5.5	Exs 1	Done	2- 4
	Exs 2	Done	4 - 7
	Exs 3	Done	7 – 8
	Exs 4	Done	8 - 9
...	...		
	...		

Self-scores:

Note: Export file to **PDF and name the file by following format:
Student ID_LABx.pdf*

Section 1.5

1. Hiện thực hóa mô hình trong ví dụ 5.3.1.2, tuy nhiên thay bằng điều kiện sau: $\text{sells} \leq \text{products} \leq \text{sells} + [4 \text{ số cuối của MSSV}]$

```
#define MAX_BUFFER 359

int products = 0;
int sells = 0;
```

Khởi tạo 2 biến products (sản phẩm tạo ra), sells (sản phẩm bán), MAX_BUFFER = 359 để thỏa điều kiện $\text{product} \leq \text{sell} + \text{MSSV}$ (23520359)

```
sem_t sem_product; // số lượng sản phẩm có thể bán
sem_t sem_space;    // số lượng chỗ còn trống để sản xuất

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

Khởi tạo 2 semaphore để đếm số lượng sản phẩm có sẵn và số chỗ trống còn lại để sản xuất sản phẩm

Khởi tạo mutex để tránh race condition khi đọc/ghi products và sells

```
void* producer(void* arg) {
    while (1) {
        sem_wait(&sem_space); // chờ nếu đã đầy (products == sells + 359)

        pthread_mutex_lock(&lock);
        products++;
        int p = products, s = sells;
        pthread_mutex_unlock(&lock);

        printf("[Producer] Produced: %d, Sold: %d\n", p, s);

        sem_post(&sem_product); // báo cho seller là có hàng
        sleep(3);
    }
    return NULL;
}
```

Hàm producer: kiểm tra số lượng sản phẩm trong kho

- Nếu đã đầy: chờ - ngưng sản xuất
- Chưa đầy: sản xuất sản phẩm, sau đó thông báo seller đã có sản phẩm

```

void* seller(void* arg) {
    while (1) {
        sem_wait(&sem_product); // chờ nếu chưa có hàng (products == sells)

        pthread_mutex_lock(&lock);
        sells++;
        int p = products, s = sells;
        pthread_mutex_unlock(&lock);

        printf("[Seller]   Produced: %d, Sold: %d\n", p, s);

        sem_post(&sem_space); // báo cho producer là có thêm chỗ trống
        sleep(2);
    }
    return NULL;
}

```

Hàm seller: kiểm tra sản phẩm có thể bán

- Nếu chưa có hàng: chờ bên kho thông báo
- Đã có hàng: tăng sản phẩm đã bán, tăng số chỗ trống trong kho, báo về cho kho

```

int main() {
    pthread_t t1, t2;

    // Khởi tạo semaphore
    sem_init(&sem_product, 0, 0); // Ban đầu chưa có sản phẩm
    sem_init(&sem_space, 0, MAX_BUFFER); // Tối đa 359 sản phẩm (products - sells <= 359)

    pthread_create(&t1, NULL, producer, NULL);
    pthread_create(&t2, NULL, seller, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    // Giải phóng tài nguyên
    pthread_mutex_destroy(&lock);
    sem_destroy(&sem_product);
    sem_destroy(&sem_space);

    return 0;
}

```

- Khởi tạo số sản phẩm ban đầu là 0, chỗ trống trong đoạn [0; 359]
- Tạo 2 tiểu trình cho producer và seller, sau đó thực hiện công việc đồng bộ đa tiểu trình
- Sau khi kết thúc có thể giải phóng tài nguyên.

***Kết quả chạy code:**

```

[Producer] Produced: 1, Sold: 0
[Seller]    Produced: 1, Sold: 1
[Producer] Produced: 2, Sold: 1
[Seller]    Produced: 2, Sold: 2
[Producer] Produced: 3, Sold: 2
[Producer] Produced: 4, Sold: 2
[Seller]    Produced: 4, Sold: 3
[Producer] Produced: 5, Sold: 3

```

2. Đồng bộ đa tiểu trình của 1 mảng a có các phần tử là số nguyên

```

#define MAX_SIZE 100

int a[MAX_SIZE];      // Mảng dùng chung
int size = 0;         // Kích thước hiện tại của mảng

```

- Khởi tạo mảng dùng chung cho 2 thread và kích thước hiện tại của mảng

1. Chưa đồng bộ

```

void* producer(void* arg) {
    while (1) {
        int num = rand() % 100;

        if (size < MAX_SIZE) {
            a[size++] = num;
            printf("[Producer] Added %d to array. Size = %d\n", num, size);
        } else {
            printf("[Producer] Array full. Skipping...\n");
        }

        usleep(100);
    }
    return NULL;
}

```

Hàm producer: tạo 1 số ngẫu nhiên

- Nếu mảng chưa đầy (< MAX_SIZE): thêm số đó vào mảng, tăng số lượng phần tử trong mảng, in ra số lượng mảng

```

void* consumer(void* arg) {
    while (1) {
        if (size > 0) {
            int index = rand() % size;
            int value = a[index];
            a[index] = a[size - 1];
            size--;
            printf("[Consumer] Removed %d from array. Size = %d\n", value, size);
        } else {
            printf("[Consumer] Nothing in array a\n");
        }

        usleep(100);
    }
    return NULL;
}

```

Hàm consumer: tạo 1 vị trí bất kì

- Nếu còn phần tử ($size > 0$): xóa 1 số tại vị trí đó, giảm số lượng mảng, in ra số lượng mảng

*** Vấn đề gặp phải:** race condition tại thời điểm cập nhật số lượng mảng (thời điểm cập nhật quá nhanh khiến biến không cập nhật kịp)

```

[Consumer] Removed 44 from array. Size = 33
[Consumer] Removed 57 from array. Size = 33
[Producer] Added 72 to array. Size = 34
[Consumer] Removed 1 from array. Size = 32
[Producer] Added 64 to array. Size = 33
[Consumer] Removed 64 from array. Size = 32

```

2. Đồng bộ

```
sem_t mutex;
```

- Khởi tạo thêm 1 biến mutex để đồng bộ

```

void* producer(void* arg) {
    while (1) {
        int num = rand() % 100;

        sem_wait(&mutex);

        if (size < MAX_SIZE) {
            a[size++] = num;
            printf("[Producer] Added %d to array. Size = %d\n", num, size);
        } else {
            printf("[Producer] Array full. Skipping...\n");
        }

        sem_post(&mutex);

        usleep(100);
    }
    return NULL;
}

```

```

void* consumer(void* arg) {
    while (1) {
        sem_wait(&mutex); // Lock mảng a

        if (size > 0) {
            int index = rand() % size; // Lấy phần tử bất kỳ
            int value = a[index];
            a[index] = a[size - 1]; // Thay thế bằng phần tử cuối để xóa
            size--;
            printf("[Consumer] Removed %d from array. Size = %d\n", value, size);
        } else {
            printf("[Consumer] Nothing in array a\n");
        }

        sem_post(&mutex); // Unlock mảng a

        usleep(100);
    }
    return NULL;
}

```

- sem_wait(&mutex): khóa truy cập vùng dữ liệu dùng chung (mảng a)
- sem_post(&mutex): mở khóa
- ≡ Đảm bảo chỉ có 1 thread được cập nhật a[] và size tại 1 thời điểm

*** Kết quả chạy code:**

```
[Consumer] Removed 86 from array. Size = 6
[Producer] Added 20 to array. Size = 7
[Consumer] Removed 34 from array. Size = 6
[Consumer] Removed 30 from array. Size = 5
[Producer] Added 88 to array. Size = 6
[Consumer] Removed 88 from array. Size = 5
[Producer] Added 96 to array. Size = 6
```

3. Hiện thực mô hình 2 process A, B

Hiện thực mô hình:

```
int x = 0;
```

- Khởi tạo biến $x = 0$;

```
void* processA(void* arg) {
    while (1) {
        x = x + 1;
        if (x == 20)
            x = 0;
        printf("A: x = %d\n", x);
        usleep(80);
    }
    return NULL;
}

void* processB(void* arg) {
    while (1) {
        x = x + 1;
        if (x == 20)
            x = 0;
        printf("B: x = %d\n", x);
        usleep(100);
    }
    return NULL;
}
```

- Xây dựng 2 hàm processA và processB

*** Kết quả chạy code:**

```

B: x = 18
A: x = 17
A: x = 18
B: x = 18
A: x = 19
B: x = 0
A: x = 1
B: x = 2

```

- ⊆ Kết quả trùng lặp (in 2 lần x = 18) → race condition

4. Đồng bộ với mutex để sửa lỗi bất hợp lý trong kết quả của mô hình Bài 3

```

int x = 0;
pthread_mutex_t lock;

```

- Khởi tạo mutex lock để chỉ có 1 tiến trình truy cập vùng dữ liệu dùng chung (biến x) trong 1 thời điểm

```

void* processA(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);

        x = x + 1;
        if (x == 20)
            x = 0;
        printf("A: x = %d\n", x);

        pthread_mutex_unlock(&lock);
        usleep(80);
    }
    return NULL;
}

void* processB(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);

        x = x + 1;
        if (x == 20)
            x = 0;
        printf("B: x = %d\n", x);

        pthread_mutex_unlock(&lock);
        usleep(100);
    }
    return NULL;
}

```

- Thêm lock và unlock vào cho từng thread

***Kết quả chạy code:**

```
A: x = 0
```

```
B: x = 1
```

```
A: x = 2
```

```
B: x = 3
```

```
A: x = 4
```

```
B: x = 5
```

```
A: x = 6
```

```
B: x = 7
```